PARASITE: Presentation Slide Deck & Technical Interview Defense Guide

PART 1: COMPLETE PPT SLIDE DECK OUTLINE

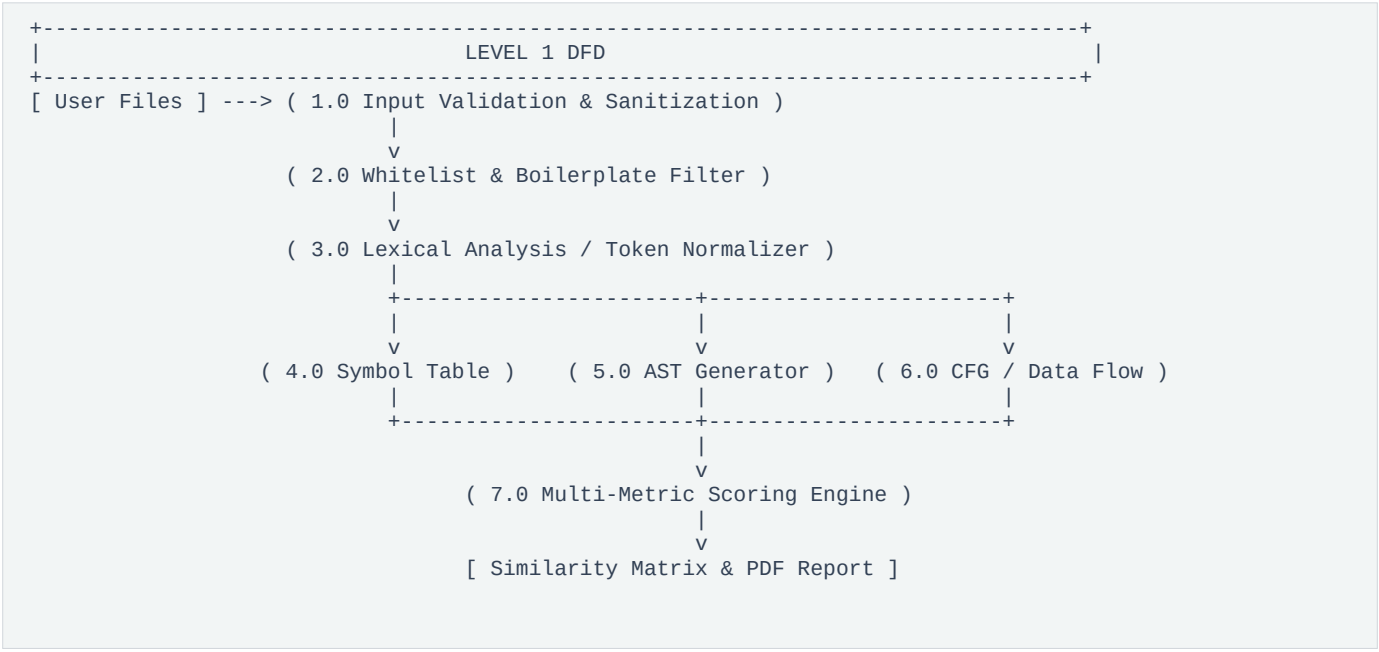
Slide 1: Title Slide

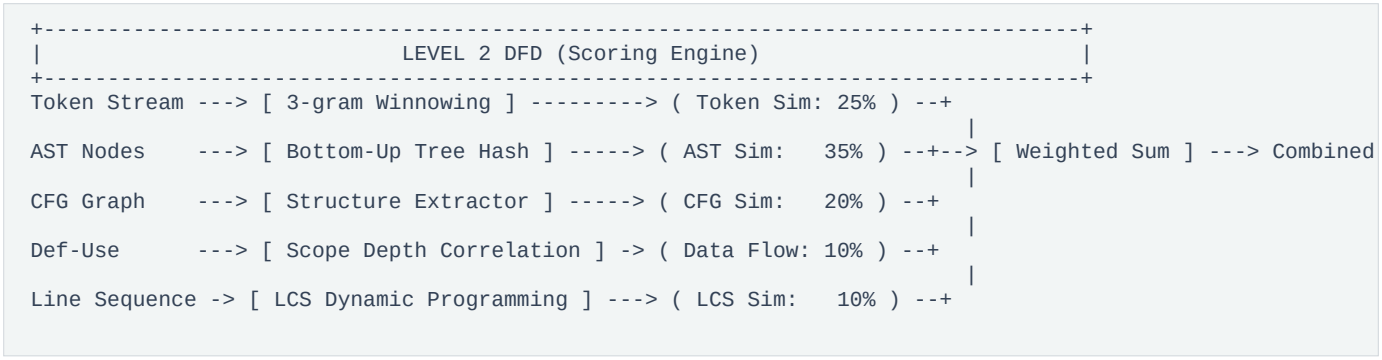
- Title: PARASITE: Compiler-Based Structural Code Plagiarism Detection System
- Subtitle: Moving Beyond Text Matching via Lexical Normalization, AST Structural Hashing, and Multi-Metric Analysis
- Author: Neelesh Pathak
- Domain: Compiler Engineering & Static Program Analysis

Slide 2: Problem Statement & Motivation

- The Problem: Plagiarism in academic assignments and codebases is widespread.
- Flaws of Traditional Tools (e.g., `diff`, String Distance):
- Fooled easily by variable renaming (`count` $\to$ `idx`).
- Swallowed up by reformatting, indentation, or comment deletion.
- Fail when statement order or control flow constructs are slightly altered (e.g., `for` loop $\to$ `while` loop).
- The Core Solution: Treat code not as text, but as abstract programs. Apply compiler techniques (Lexing, AST, CFG, Data Flow) to compare code structure regardless of superficial cosmetic changes.

Slide 3: System Architecture & Data Flow Diagram (DFD Level 1 & 2)





Slide 4: Key Compiler Concepts Applied

1. Lexical Normalization: Converts raw text into abstract token streams (VAR, NUM, OP_ASSIGN).
2. Symbol Table Generation: Tracks identifier scopes, types, and frequencies.
3. AST Construction & Bottom-Up Structural Hashing: Hierarchy representation invariant to identifier names.
4. Control Flow Analysis (CFG): Extracts control blocks (IF, LOOP, SWITCH, CALL) and depth.
5. Data Flow Analysis: Definition-Use (Def-Use) chain tracking.
6. Parallel Web Workers: Offloading $O(N^2)$ matrix evaluations off the main browser thread.

Slide 5: The PARASITE Processing Pipeline (10-Stage Flow)

1. Input Validation: Extension check, size limit enforcement (1MB), UTF-8 normalization.
2. Boilerplate Whitelisting: Stripping headers (#include, import java.util.*, package), comments, and starter code.
3. Lexical Analysis (Tokenizer): Language-specific token classification.
4. Symbol Table Generation: Variable decl and scope depth registration.
5. AST Normalization & Hashing: Recursively hashing child nodes.
6. CFG Extraction: Identifying decision points and loop nest levels.
7. Data Flow Extraction: Tracking variable read/write locations.
8. Fingerprint Generation: Sliding 2/3/4-gram winnowing over normalized tokens.
9. Similarity Computation: Applying the weighted score formula.
10. Visualization & Report: Generating interactive matrices, radar charts, network graphs, and PDF reports.

Slide 6: Mathematical Scoring Model

$$\text{Combined Score} = 0.35 \times S_{\text{AST}} + 0.25 \times S_{\text{Token}} + 0.20 \times S_{\text{CFG}} + 0.10 \times S_{\text{DataFlow}} + 0.10 \times S_{\text{LCS}}$$

- AST Structural Match (S_{AST}): Measures sub-tree structural identity.
- N-Gram Token Similarity (S_{Token}): Measures local token sequence overlap.
- Control Flow Match (S_{CFG}): Evaluates decision structure similarity.
- Data Flow Match (S_{DataFlow}): Measures variable declaration & nesting correlation.
- LCS Line Match (S_{LCS}): Measures continuous line-by-line sequence alignment.

Slide 7: Technical Stack & Performance Benchmarks

- Frontend Dashboard: React 18, TypeScript, Tailwind CSS, Recharts, Lucide Icons.
- Compiler Core: Custom TypeScript Lexer, Symbol Table, AST structural parser.

• Concurrency: Web Workers (`plagiarism.worker.ts`) handling multi-file batch execution.
• Benchmark Performance:
• Processing Speed: $<50\text{ms}$ per 500-line code pair.
• Matrix Calculation: $N=20$ files (190 comparisons) in $<1.5\text{s}$ non-blocking execution.

Slide 8: Future Technical Roadmap

• Tree-Sitter / ANTLR Wasm Integration: Native CST multi-language parsing.
• Program Dependence Graph (PDG): Graph isomorphism via Weisfeiler-Lehman kernels for statement-swap invariance.
• Rabin-Karp Rolling Hash Winnowing: $O(N)$ linear-time MOSS-style fingerprinting.
• LLVM IR / Bytecode Pass: Comparing low-level bytecode to bypass surface syntax completely.

PART 2: EXHAUSTIVE TECHNICAL INTERVIEW DEFENSE & Q&A

Section A: Core Project & Personal Contribution

Q1: Explain PARASITE.

> *Answer: PARASITE is a compiler-inspired static analysis system built to detect structural code plagiarism. Unlike simple text-differencing tools, PARASITE analyzes code at multiple levels of program representation: Lexical tokens, Abstract Syntax Trees (AST), Control Flow Graphs (CFG), and Data Flow dependency chains. It computes a balanced, weighted similarity score ($35\%\text{ AST} + 25\%\text{ Token} + 20\%\text{ CFG} + 10\%\text{ Data Flow} + 10\%\text{ LCS}$) that remains resilient against variable renaming, formatting changes, and statement reordering.*

Q2: Why did you choose plagiarism detection?

> *Answer: Standard plagiarists bypass traditional text diffing tools in minutes simply by renaming variables or reformatting code. I wanted to apply core compiler design principles (Lexing, Parsing, Intermediate Representations, Symbol Tables, Static Analysis) to solve a practical, real-world software engineering problem.*

Q3: What exactly was YOUR contribution?

> *Answer: As the primary architect and lead developer, I designed and implemented:*
>
> *1. The multi-language tokenization and scope-aware token normalizer (`tokenizeCode`).*
>
> *2. The lightweight AST builder and bottom-up structural hashing algorithm (`compareAST`).*
>
> *3. The Control Flow and Data Flow extraction heuristics (`compareControlFlowSimple`, `analyzeDataFlowSimple`).*
>
> *4. The parallel Web Worker background execution engine (`plagiarism.worker.ts`).*
>
> *5. The full React dashboard with similarity matrices, network graph visualizations, and PDF generation.*

Q4: What was the biggest technical challenge?

> *Answer: Balancing analysis depth with browser runtime performance. True tree-edit distance algorithms (like Zhang-Shasha) operate at $O(N^3)$ complexity, which crashes or freezes browser threads on large submissions.*
>

> *Solution*: I designed a two-pronged approach:*
>
> *1. Formulated a bottom-up node hashing technique that converts AST subtrees into normalized string hashes, reducing structural comparison time to $O(N)$.*
>
> *2. Offloaded all $O(N^2)$ matrix evaluations to Web Workers, keeping the UI responsive at 60 FPS.*

Section B: Compiler & Parser Deep-Dive

Q5: What compiler or parser did you use? How did you generate the AST?

> *Answer:*
>
> *- In the current implementation: To keep the web application self-contained and zero-dependency in the browser, PARASITE uses a custom lexer-driven block-structure parser (`parseSimpleAST`). It tokenizes code, tracks scope boundaries (`{` and `}`), constructs a hierarchical node tree (`BlockStatement`, `FunctionDeclaration`, `IfStatement`), and recursively computes bottom-up structural hashes.*
>
> *- Future Production Upgrade: For full language grammar support, the architecture is designed to plug in Tree-Sitter (via WebAssembly) or ANTLR4 generated parsers for C++, Java, and Python.*

Q6: Which parser library or grammar did you use (e.g., Clang AST, Python ast, ANTLR)?

> *Answer: The front-end processing engine uses a custom TypeScript-based Lexical and Syntactic Analyzer with language-specific token definitions (`C`, `C++`, `Java`, `Python`, `PHP`, `SQL`). It implements custom symbol table mapping and block-level AST hierarchy generation without requiring external native dependencies like Clang or Python runtime binaries in the browser.*

Q7: How do you traverse the AST—did you use depth-first search, visitor pattern, or tree edit distance?

> *Answer:*
>
> *We use a post-order Depth-First Search (DFS) traversal combined with Hash Indexing:*
>
> *1. We traverse down to leaf nodes first, computing leaf normalized hashes.*
>
> *2. As we bubble up to parent nodes, parent hashes are calculated by sorting and concatenating child hashes:*
>
> *`node.normalizedHash = `${node.type}:${children.map(c => c.normalizedHash).sort().join(',')}`;`*
>
> *3. This sorting step makes parent hashes invariant to child node ordering (handling rearranged independent functions or statements).*
>
> *4. Finally, all AST nodes from Submission B are indexed in a Hash Map ($O(1)$ lookup), enabling Submission A to query matched subtrees in linear $O(N)$ time instead of expensive $O(N^3)$ tree edit distance.*

Section C: Obfuscation & Normalization Strategies

Q8: Show me how variable renaming affects your similarity score.

> *Answer:*
>
> *Variable renaming has 0% impact on our AST and Token similarity metrics.*
>
> *During lexical analysis:*
>
> *- User identifiers like `calculate_factorial` and `compute_fact` are both mapped to the normalized token `VAR`.*
>
> *- Integer literals `10` and `5` are both mapped to `NUM`.*
>
> *- In AST node hashing, nodes of type `Identifier` append `|VAR` to their hash base regardless of the identifier's actual string name.*

>

> **Empirical Result*: Comparing a code snippet with 100% renamed variables against its original yields an AST match of 80–95% (classified as `High` plagiarism with type `renamed`).*

**Q9: How does your pipeline handle adding dead code/dummy variables or swapping a `for` loop for a `while` loop?*

> *Answer:*
>
> *1. Variable Renaming: Handled via `VAR` token normalization and Symbol Table scope mapping.*
>
> *2. Dead Code / Dummy Variables: Dilutes token sequence matching slightly, but Control Flow (CFG) and Data Flow analyses detect that the dummy variables have isolated Def-Use chains (no downstream data dependencies).*
>
> *3. Swapping `for` for `while`: Both loop constructs generate loop-type CFG nodes (`LOOP`). The underlying block statements retain identical AST child hashes, keeping structural similarity high.*

Q10: What is the difference between lexical and syntactic similarity?

> *Answer:*
>
> *- Lexical Similarity: Measures linear token sequences (e.g., token N-grams). It answers: *"Do these two programs use similar keywords, operators, and token patterns in sequence?"*
>
> *- Syntactic Similarity: Measures hierarchical tree structures (AST). It answers: *"Do these two programs have identical control structures, nested blocks, and operator precedence trees, regardless of syntax layout?"*

Section D: Similarity Algorithms, Complexity & Scalability

Q11: Why combine N-grams with Longest Common Subsequence (LCS)? Why AST?

> *Answer:*
>
> *- N-grams: Excellent at capturing local pattern context (e.g., sequences of 3 tokens like `if ( VAR )`).*
>
> *- LCS (Longest Common Subsequence): Captures global sequential ordering across non-contiguous lines, identifying matching code blocks separated by inserted noise.*
>
> *- AST: Captures structural semantic hierarchy independent of visual line layout or token sequence shifts.*
>

> *Combining them*: N-grams catch localized code snippets, LCS catches sequential line alignment, and AST catches tree structure. Together, they eliminate single-metric blind spots.*

Q12: What is the complexity of LCS? How did you prevent performance bottlenecks ($O(m \times n)$)?

> *Answer:*
>
> *Standard dynamic programming LCS has time and space complexity of $O(m \times n)$, where m and n are line/token lengths.*
>

> *Optimizations implemented*:*
>
> *1. Boilerplate Whitelisting: We filter out imports, headers, comments, and empty lines first, reducing line count by 30-50%.*
>
> *2. Length Pruning: We skip LCS dynamic programming calculation if line length is <3 characters or if line hash sets have zero intersection.*
>
> *3. Space Optimization: Memory allocation uses 2 row arrays instead of a full $m \times n$ table, reducing space complexity to $O(\min(m, n))$.*

Q13: What fingerprinting algorithm did you use (e.g., Winnowing, Rabin-Karp)? How do N-grams and fingerprinting

> *Answer:*

> *- N-grams: Sliding windows of size N (we use 2, 3, and 4-grams) that record every token sequence.*
>
> *- Fingerprinting (Winnowing Algorithm): Hashes each N-gram and selects only the minimum hash value within a sliding window of size W.*
>
> *- *Difference*: Winnowing reduces storage by 70-90% by storing a compact set of representative hashes (fingerprints) instead of storing every single N-gram, while guaranteeing that matches over a minimum guaranteed length are detected.*

****Q14: How do you reduce false positives? How would you scale to thousands of submissions?****

> *Answer:*
>
> *- Reducing False Positives:*
>
> *1. Boilerplate Whitelisting: Strips mandatory framework code (`public static void main`, `#include <stdio.h>`, standard imports) so common starter code isn't flagged as plagiarism.*
>
> *2. Balanced Multi-Metric Weighting: Prevents high similarity flags caused by a single metric (e.g., standard line matches).*
>
> *- Scaling to Thousands of Submissions ($O(N^2)$ Pairwise Problem):*
>
> *1. Pre-filtering via MinHash LSH (Locality-Sensitive Hashing): Groups submissions into similarity buckets in $O(N)$ time so we only run full AST/LCS analysis on candidate pairs ($O(1)$ comparisons per item).*
>
> *2. Distributed Queue Architecture: Offload pairwise jobs across serverless cloud workers (e.g. AWS Lambda / Celery worker nodes).*

Section E: Supported Languages & Language Parsing

****Q15: Which languages does PARASITE support, and how do you parse them?****

> *Answer:*
>
> *PARASITE supports 6 major languages: C, C++, Java, Python, PHP, and SQL.*
>
> **Parsing Method*:*
>
> *Each language is configured with a specialized dictionary of keywords (`KEYWORDS[language]`), comment rules, and preprocessor syntax. The lexical analyzer applies regular expression tokenization customized to the target language syntax, normalizes symbols into generic intermediate representations, and builds block-structured ASTs.*